



MongoDB Session 2 Cheat Sheet: Working with MongoDB Databases

Date: July 14, 2025

Platform: MongoDB Compass Shell (`mongosh`)

This cheat sheet summarizes Session 2 topics for MongoDB operations using raw shell commands in MongoDB Compass. Each topic includes a brief explanation and an example command with expected output. Run commands in the Compass **Shell** tab and verify changes in the **Collections** tab or with `show dbs / show collections` .

1. Data Types in MongoDB

Explanation: MongoDB uses BSON (Binary JSON) to store data types like String, Integer, Double, Boolean, Array, Object, Date, ObjectId, Null, etc. Documents in a collection can have varied types without a fixed schema.

Example:

```
use school
db.students.insertOne({
  name: "Alice Smith",           // String
  age: 22,                       // Integer
  grades: ["A", "B+"],          // Array
  address: { city: "New York" }, // Object
  enrolled: new Date(),         // Date
  active: true                   // Boolean
})
```

Output:

```
{ acknowledged: true, insertedId: ObjectId("...") }
```

2. Databases and Collections

Explanation: A database holds collections, and a collection stores documents (like a table in relational databases). Databases and collections are created implicitly when data is added or explicitly with commands.

Example:

```
use school
db.createCollection("courses")
```

Output:

```
{ ok: 1 }
```

3. Create Database

Explanation: Use `use <dbName>` to switch to a database, creating it implicitly. It persists only after adding data (e.g., a document or collection).

Example:

```
use library
db.books.insertOne({ title: "MongoDB Basics", year: 2023 })
```

Output:

```
{ acknowledged: true, insertedId: ObjectId("...") }
```

Verify: `show dbs` (lists `library`)

4. Create Collection

Explanation: Create a collection explicitly with `db.createCollection()` or implicitly by inserting documents into a non-existent collection.

Example:

```
use school
db.createCollection("teachers")
```

Output:

```
{ ok: 1 }
```

5. Insert Document

Explanation: Add documents to a collection using `insertOne()` (single document) or `insertMany()` (multiple). Each document gets a unique `_id` (ObjectId if not specified).

Example:

```
use school
db.students.insertOne({ name: "Bob Johnson", age: 19, grade: "A-" })
```

Output:

```
{ acknowledged: true, insertedId: ObjectId("...") }
```

6. Query Document

Explanation: Use `db.collection.find(query, projection)` to retrieve documents. Query

filters documents (e.g., `{ age: 19 }`), and projection selects fields (e.g., `{ name: 1, _id: 0 }`). Add `.pretty()` for readable output.

Example:

```
use school
db.students.find({ age: { $gte: 20 } }, { name: 1, grade: 1, _id: 0 }).pretty()
```

Output:

```
{ "name": "Alice Smith", "grade": "B" }
```

7. Update Document

Explanation: Modify documents with `updateOne()` or `updateMany()`. Use operators like `$set` to update fields or `$inc` to increment numbers.

Example:

```
use school
db.students.updateMany({ name: "Alice Smith" }, { $set: { grade: "A+" } })
```

Output:

```
{ acknowledged: true, matchedCount: 1, modifiedCount: 1 }
```

8. Delete Document

Explanation: Remove documents with `deleteOne()` or `deleteMany()` based on a query. Deletions are permanent, so back up data if needed.

Example:

```
use school
db.students.deleteMany({ name: "Bob Johnson" })
```

Output:

```
{ acknowledged: true, deletedCount: 1 }
```

9. Drop Collection

Explanation: Use `db.collection.drop()` to delete a collection and all its documents. This is irreversible.

Example:

```
use school
db.courses.drop()
```

Output:

```
true
```

10. Drop Database

Explanation: Use `db.dropDatabase()` to delete the current database and all its collections. Use cautiously, as this is permanent.

Example:

```
use school
db.dropDatabase()
```

Output:

```
{ ok: 1 }
```

Quick Tips

- **Verify:** Use `show dbs` (databases) or `show collections` (collections) to check your work.
- **Visual Check:** Use Compass's **Collections** tab to see databases/collections/documents.
- **Errors:** If a command fails (e.g., dropping a non-existent collection), expect `false` or an error.
- **Clear Shell:** Click the trash icon in Compass Shell to clear output.
- **Permissions:** Ensure your MongoDB user has create/write/drop permissions for remote servers.

Troubleshooting

- **Connection:** Check the connection string in Compass's **Connect** dialog.
- **No Output:** Confirm the correct database (`use <dbName>`) and collection exist.
- **Syntax:** Ensure commands are copied exactly (e.g., no missing brackets).

End of Cheat Sheet